

1.4 Hardware Consequences and Processing Latency Limits

► LEARNING OBJECTIVE

Rather than treating hardware latency as an arbitrary processing delay, this section grounds timing in physical acoustic accumulation, contrasts on-chip Block RAM allocations against external memory bandwidth, and exposes why the migration of audio front-ends to FPGA logic is driven by deterministic execution and zero-jitter hardware isolation rather than raw arithmetic throughput.

Organic Bridge from Section 1.3: Mapping Equations onto Physical Transistors. In Section 1.3, we completed the mathematical microscope of the front-end DSP pipeline, transforming 400 streaming pressure samples into 80 Log-Mel spectral values. But an algorithm on paper assumes infinite memory and zero execution time. In this section, we force these abstract equations to collide with physical silicon: evaluating how the mathematical parameters translate into memory footprint and processing deadlines across an Edge GPU (NVIDIA Jetson Orin Nano) and a spatial FPGA fabric (AMD Xilinx Kria KV260).

State 1: The Physical Latency Floor

Latency in a streaming audio system has an absolute physical lower bound dictated by acoustics before any semiconductor chip is ever powered on. Three temporal quantities are permanently locked by the front-end acoustic specification:

Parameter	Value	Mathematical Origin & Physical Definition
First-frame accumulation floor	25 ms	$L/f_s = 400/16,000$ (Microphone diaphragm collection time)
Hop cadence between frames	10 ms	$H/f_s = 160/16,000$ (Periodic emission interval to downstream model)
Overlap ratio across frames	60%	$(L-H)/L = 240/400$ (Shared historical context preserving energy)

This acoustic reality establishes a strict latency floor that no processor can circumvent: **Frame 0 physically cannot exist** before 25 ms of continuous air pressure vibrations have physically struck the microphone diaphragm. Even a theoretical 5 GHz CPU executing the spectral transformation in zero nanoseconds cannot emit the first feature vector at $t = 24.9$ ms because the four-hundredth audio sample has not yet occurred in physical reality.

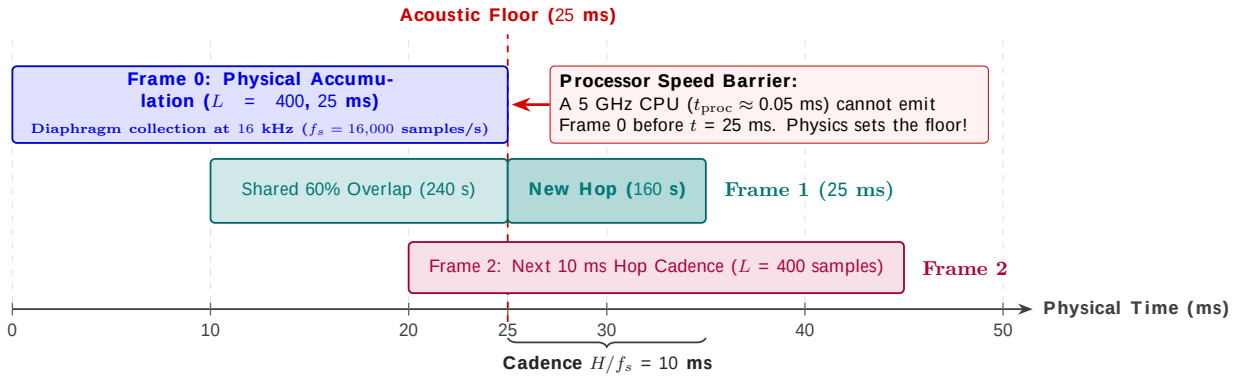


Figure 1.4a: The acoustic latency floor versus processor compute speed. (1) **What it is:** Streaming audio timeline showing physical frame accumulation and periodic hop emission. (2) **What to look at:** Frame 0 requires a full 25 ms (400 samples) of acoustic pressure accumulation before processing can begin; a 5 GHz CPU executing in near-zero time cannot bypass this acoustic floor. Subsequent frames advance every 10 ms (160 samples) with a 60% (240-sample) overlap. (3) **What it proves:** Real-time latency in voice edge systems is strictly governed by physical wave propagation and windowing acoustics, not processor clock frequency.

Once the initial 25 ms accumulation floor is satisfied, the system enters steady-state operation. From this point forward, the microphone emits a new block of $H = 160$ samples every 10 ms. This 10 ms interval is the **hard real-time deadline**: all downstream computations—windowing, 512-point FFT, power spectrum, 80-channel Mel filtering, logarithmic compression, and neural network inference—must complete within 10 ms. A faster processor does not shrink the 25 ms physical floor; it merely prevents additional computational delay from accumulating on top of it.

State 2: Memory Footprint on Physical Silicon

To evaluate physical feasibility, we ground our memory requirements against official hardware specifications from the AMD DS890 datasheet for the Zynq UltraScale+ **XCK26** MPSoC deployed on the Kria KV260: **117,120** logic cell Lookup Tables (LUTs); **144** Block RAM blocks of 36 kbit each (5.1 Mb $\approx$ 648 KiB total on-chip SRAM); **64** UltraRAM blocks (18.0 Mb $\approx$ 2.25 MiB total bulk SRAM); and **1,248** DSP48E2 compute slices optimized for single-cycle multiply-accumulates.

Now contrast these available resources against the two core state structures that the streaming DSP front-end must maintain in 32-bit single-precision floating point (FP32):

- **Audio Ring Buffer (400 samples):** At 4 bytes per sample, storing running acoustic history requires $400 \times 4 \text{ B} = 1,600$ bytes. A single 36-kbit Block RAM block provides 4,608 bytes of dual-port storage. The entire time-domain state occupies only 34.7% **of a single BRAM block** (0.2% of total on-chip BRAM).
- **Mel Filterbank Weight Matrix (80×257):** Dense storage of triangular filter weights requires $80 \times 257 \times 4 \text{ B} = 82,240$ bytes = 80.3 KiB. This entire coefficient matrix requires 18 Block RAM blocks, consuming exactly 12.4% **of total on-chip BRAM**.

► PHYSICAL INSIGHT

Because the combined memory footprint of the front-end state and filterbank matrix (81.9 KiB) consumes less than 13% of on-chip Block RAM, the entire DSP pipeline fits within local FPGA SRAM. The hardware requires zero round-trips to external DDR DRAM, eliminating dynamic memory arbitration, DRAM refresh stalls, and memory bus energy consumption.

State 3: The Arithmetic Fallacy — Throughput vs. Deterministic Isolation

Evaluating the 80-channel Mel filterbank requires multiplying 257 power spectral bins by 80 triangular weight vectors:

$$\text{Workload per frame} = 80 \times 257 = 20,560 \text{ MAC operations}$$

At a hop cadence of 10 ms, the front-end produces 100 frames per second, resulting in an aggregate computational throughput of:

$$\text{Throughput} = 20,560 \text{ MAC/frame} \times 100 \text{ frames/s} = 2.056 \times 10^6 \text{ MAC/s} \approx 2.06 \text{ MMAC/s}$$

Spread evenly across the 1,248 DSP48E2 slices of the Kria KV260, each individual DSP slice would need to execute only 1,647 **operations per second**. When clocked at 200 MHz, a single DSP48E2 slice provides 200×10^6 MAC/s; thus, the entire front-end arithmetic workload represents less than 0.001% of the chip’s computational capacity.

Now compare this to the competing edge accelerator: the **NVIDIA Jetson Orin Nano Super**. The Orin Nano delivers up to **67 sparse INT8 TOPS** (or approximately **33 dense TOPS** at a 25 W power ceiling). Against this massive computing budget, the 2.06 MMAC/s required by the voice front-end is literally **one millionth** of the GPU’s available processing bandwidth.

This disparity exposes a central architectural truth: **migrating an audio front-end to FPGA logic is never about raw arithmetic throughput**. An edge GPU possesses orders of magnitude more raw floating-point performance. The migration is driven entirely by **temporal determinism and hardware isolation**:

- **The Shared Software Trap (Edge GPU):** On a GPU/CPU system, the audio front-end shares the 10 ms processing window with the Linux operating system kernel, microphone DMA driver interrupts, background network stacks, and concurrent neural network inference. Each CUDA kernel invocation incurs 5–15 μs of launch overhead, and operating system thread scheduling introduces random preemption. A deadline shared among multiple asynchronous software layers is inherently subject to latency jitter (Δt_{jitter}).
- **The Dedicated Spatial Fabric (FPGA):** On an FPGA, the DSP pipeline is constructed from dedicated, physically isolated silicon wires and registers. It does not share a clock, an execution pipeline, or a memory bus with an operating system. The front-end completes each frame in a mathematically fixed number of clock cycles (N_{clk}), guaranteeing an execution jitter of identically zero nanoseconds ($\Delta t = 0 \text{ ns}$).

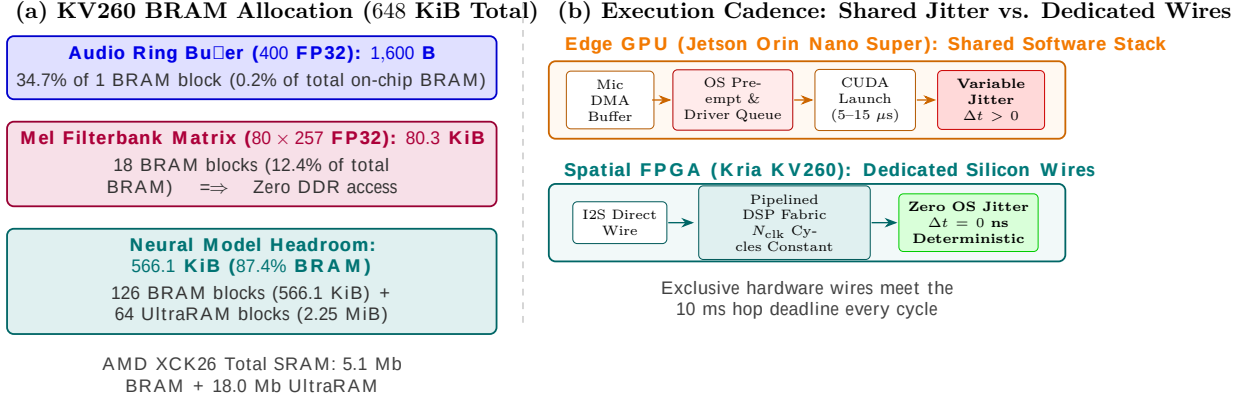


Figure 1.4b: KV260 BRAM memory breakdown and datapath determinism. (1) **What it is:** Block RAM resource distribution on the AMD XCK26 MPSoC alongside architectural pipeline comparison. (2) **What to look at:** In (a), front-end state (1,600 B) and Mel weights (80.3 KiB) occupy only 12.6% of on-chip SRAM, leaving 87.4% for acoustic neural networks. In (b), edge GPU shares the 10 ms hop window with OS kernel interrupts and memory contention (jitter), whereas the FPGA datapath executes with dedicated silicon wires and zero jitter. (3) **What it proves:** The migration imperative from GPU to FPGA is governed by deterministic latency isolation, not raw FLOPS.

★ GROUNDBREAKING FINDING

The FPGA does not outperform the edge GPU on raw arithmetic throughput; it outperforms the GPU on deterministic isolation. Dedicated silicon wires guarantee zero-jitter execution within the non-negotiable 10 ms hop deadline, completely decoupled from operating system preemption and memory contention.

Organic Bridge to Section 1.5: Stress-Testing Hardware Boundaries. We have established that the streaming front-end pipeline maps with exceptional efficiency onto physical FPGA resources—consuming minimal Block RAM and a negligible fraction of DSP slices while securing absolute latency determinism.

However, mathematical formulations reveal catastrophic points of failure when pushed beyond their operating boundaries. What happens if a designer attempts to lower latency by cutting window length in half? What happens if audio is sampled at studio rates (48 kHz)? What happens if dense matrix evaluation is chosen over sparse compression? In Section 1.5, we subject our theoretical models to four rigorous diagnostic stress-tests, examining where edge silicon breaks when pushed to its limits.